

Quora Question Pairs

Deliverable 1 — Natural Language Processing

1. Introduction

This deliverable tackles the Quora Question Pairs Kaggle challenge: given two questions, decide whether they are semantically equivalent (a duplicate) or not. Following the brief, we present **two solutions**: a simple baseline that highlights the limitations of a naive approach, and an improved model built on a hand-crafted feature vector in which several similarity functions have been implemented from scratch.

The dataset is loaded from `$HOME/Datasets/QuoraQuestionPairs/quora_data.csv` and split into train / validation / test using the exact code given in the deliverable PDF (`random_state=123`, two consecutive 5% splits), so every team works on the same partitions. All randomness in our code uses the same seed.

2. Simple Solution — TF-IDF + Logistic Regression

Following the slide *Baseline pipeline: TF-IDF + Logistic Regression* from lecture 3, our baseline concatenates the two questions with a [SEP] token, fits a TF-IDF vectoriser on the resulting strings (uni- and bi-grams, sub-linear TF, L2-normalised), and feeds the sparse vectors to a logistic-regression classifier. The implementation lives in `utils.fit_baseline`.

2.1 Limitations and observed errors

Even before running it, the baseline has three structural weaknesses: **(a)** concatenating the two questions destroys the *pairwise* structure — the model sees a single long bag of words, so it cannot reason about *which* tokens are shared and which are not; **(b)** bag-of-words discards word order, so “the cat eats the dog” and “the dog eats the cat” are encoded identically; **(c)** TF-IDF cannot bridge synonyms — *car* and *automobile* are completely orthogonal directions in the vocabulary space.

Empirically, on the validation set we find the baseline makes two characteristic kinds of mistake: **false positives** on pairs that share generic words (“*What is the meaning of life?*” / “*What is the meaning of death?*”) and **false negatives** on true paraphrases that use different vocabulary (“*How can I lose weight?*” / “*What is an effective diet plan?*”).

2.2 Features that should help

These limitations point directly at the features we add in the improved model: **set-based overlap** measures (Jaccard, word overlap) capture pairwise vocabulary sharing; **edit distance** on tokens is order-aware; **TF-IDF cosine** between the two questions (rather than over their concatenation) directly measures their pairwise topical similarity; **length-based** features capture asymmetry that bag-of-words ignores.

3. Improved Solution — Engineered Features + XGBoost

For each pair (q_1, q_2) we build a 19-dimensional feature vector and feed it to an XGBoost classifier (gradient-boosted trees). The feature vector is the heart of the deliverable; every group member owns a block of it and implements those features **from scratch** (no `nlTK`, no `scikit-learn` primitive that already does the job).

3.1 Feature vector

Block (owner)	Feature	Brief description
Lexical (Member A)	jaccard	$ A \cap B / A \cup B $ over token sets
	jaccard_content	Jaccard after stop-word removal
	word_overlap_ratio	$ A \cap B / \min(A , B)$
	common_words	Raw $ A \cap B $
	len{1,2}_tok / _char	Token and char lengths of each question
	len_diff_*, len_ratio_*	Absolute and ratio length asymmetry
Edit distance (Member B)	char_edit	Levenshtein on characters (from scratch)
	char_edit_norm	char_edit / max(len)
	tok_edit	Levenshtein on token lists
	tok_edit_norm	tok_edit / max(token len)
	common_prefix	# leading shared tokens
TF-IDF (Member C)	tfidf_cosine_scratch	Cosine of L2-normalised TF-IDF vectors, hand-coded
	tfidf_cosine_sklearn	Same value via sklearn (cross-check)

3.2 Implementation notes

Levenshtein distance. Implemented with the classical dynamic-programming recurrence $d[i,j] = \min(d[i-1,j]+1, d[i,j-1]+1, d[i-1,j-1]+c)$ where $c = 0$ if $s_i=t_j$ and 1 otherwise, but using a *two-row rolling table*, dropping memory from $O(mn)$ to $O(\min(m,n))$. The same function is reused at the character and the token level.

TF-IDF from scratch. `TfidfFromScratch` reproduces the formulation used in lecture 3: optional sub-linear TF ($1 + \log tf$), smooth IDF $\log((1+N)/(1+df)) + 1$, and a final L2 normalisation, after which cosine similarity reduces to a plain dot product (slide “*With L2-normalised vectors, cosine reduces to a dot product*”). We feed both our cosine and the sklearn cosine to the classifier; comparing the two values per row is our sanity check.

Jaccard variants. Operating on token sets rather than multisets, with and without stop-word removal. Stop words inflate raw overlap of unrelated questions (both contain *what, is, the*), so the stop-word-free version is a much sharper signal.

3.3 Classifier

We use XGBoost with 400 trees, `max_depth = 6`, `learning_rate = 0.1`, sub-sampling 0.9, and the same `random_state` as the rest of the project. Gradient-boosted trees are a natural fit here: the feature vector is small, dense and numerical, with strong non-linear interactions (e.g. a small edit distance is informative *only* when the questions are not trivially short).

4. Results

Metrics are computed by `reproduce_results.ipynb`, which loads both saved models and produces a single dataframe with **ROC-AUC**, **precision** and **recall** on each split. The numbers below are placeholders: please re-run the notebook on the real dataset and copy the produced table here before submitting.

Model	Split	ROC-AUC	Precision	Recall
-------	-------	---------	-----------	--------

Baseline (TF-IDF + LogReg)	train	—	—	—
	val	—	—	—
	test	—	—	—
Improved (features + XGB)	train	—	—	—
	val	—	—	—
	test	—	—	—

Expected pattern: the improved model should beat the baseline by 4–8 ROC-AUC points, with the largest relative gains on validation/test where the baseline overfits the training vocabulary. The most informative engineered features are typically the two TF-IDF cosine columns and jaccard_content; tok_edit_norm contributes meaningfully on shorter questions.

5. Reproducibility

To reproduce our results from a fresh checkout:

```
conda env create -f environment.yml --name quora_test_env
conda activate quora_test_env
# place quora_data.csv into $HOME/Datasets/QuoraQuestionPairs/
jupyter nbconvert --to notebook --execute reproduce_results.ipynb
```

train_models.ipynb trains both models and writes them to `./models/`; if the folder already contains the expected files the notebook is a no-op (per the deliverable spec). **reproduce_results.ipynb** only loads the saved models, builds the feature matrix on the three splits, and produces the metric dataframe. It runs in well under 10 minutes and trains nothing.

6. Distribution of work

The team is composed of three members. The work has been split so that no two members coded the same thing twice.

Member	Owned section in utils.py	Documentation notebook
Member A	Lexical / set-based features (jaccard, overlap, length)	utils_MemberA.ipynb
Member B	Edit-distance features (Levenshtein from scratch)	utils_MemberB.ipynb
Member C	TF-IDF cosine similarity from scratch (TfidfFromScratch)	utils_MemberC.ipynb

All three members are jointly responsible for `train_models.ipynb`, `reproduce_results.ipynb`, the data-loading and evaluation helpers in `utils.py`, and this report. Each member ran both notebooks end-to-end on a clean environment built from `environment.yml` to confirm reproducibility.

7. Repository layout

```
Name1_Name2_Name3.zip
|-- main.pdf                <-- this file
|-- environment.yml
|-- utils.py                shared functions (organised by member)
|-- train_models.ipynb     trains and saves baseline + improved
|-- reproduce_results.ipynb loads models, computes metrics
|-- utils_MemberA.ipynb    per-member function explanation
|-- utils_MemberB.ipynb
|-- utils_MemberC.ipynb
`-- models/                created by train_models.ipynb
```

```
|-- baseline.joblib  
|-- improved.joblib
```